ALLIANCE UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

Database Management System

Assignment 4

RELATIONAL DATABASES vs NoSQL DATABASES

Name	Razin Basha R
Roll No	2511021240012
Course	Database Management System
Programme	BCA AI/ML
Semester	III

1. Introduction

Every application today, from a small college app to a giant shopping website, needs a place to store its data. That place is a database. For a long time the relational database was the default choice for almost everything. Then, as websites grew bigger and started handling millions of users and many different kinds of data, a new family of databases called NoSQL became popular.

In my earlier assignment I designed a Cab Booking System using a relational database with five tables: Customer, Driver, Car, Booking and Payment. While doing that work, I wondered whether a relational database would still be the best choice if the cab app became as big as a real company. This report is my attempt to answer that question. It compares relational and NoSQL databases on four points, which are scalability, consistency, performance and use cases. It also explains why different organizations pick different database technologies, with real-world examples.

2. Relational Databases

A relational database stores data in tables made of rows and columns. Each table has a primary key, and tables are connected to each other using foreign keys. The data is accessed and managed using SQL (Structured Query Language). Because the structure (schema) is fixed in advance, every row in a table follows the same format.

Main features:

- Fixed schema defined before data is stored.
- Relationships between tables using primary and foreign keys.
- Powerful queries with SQL, including JOINs across several tables.
- ACID properties: Atomicity, Consistency, Isolation and Durability, which keep transactions safe and reliable.

Popular examples: MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server and SQLite.

3. NoSQL Databases

NoSQL stands for "Not Only SQL". These databases do not depend on the fixed table structure of relational systems. They are designed to handle very large amounts of data, many different data formats, and heavy traffic spread over many servers. Most NoSQL databases have a flexible schema, so each record can have different fields.

There are four main types of NoSQL databases:

Type	How it stores data	Examples
Document	Stores data as JSON-like documents, where each document can have its own fields.	MongoDB, CouchDB
Key-Value	Stores data as simple key and value pairs, like a big dictionary. Very fast lookups.	Redis, Amazon DynamoDB
Column-Family	Stores data in columns grouped into families, which is good for huge amounts of data written very quickly.	Apache Cassandra, HBase
Graph	Stores data as nodes and relationships (edges), which is good for highly connected data.	Neo4j, Amazon Neptune

Most NoSQL systems follow the BASE approach (Basically Available, Soft state, Eventually consistent) instead of strict ACID. This means they favour availability and speed, and allow the data on different servers to match up a little later.

4. Comparison on the Four Key Points

4.1 Scalability

Scalability means how well a database can grow when the number of users and the amount of data increases. There are two ways to do this. Vertical scaling means making one server more powerful by adding more CPU, RAM or storage. Horizontal scaling means adding more servers and spreading the data across them.

Relational: Relational databases scale vertically very easily, but there is a limit to how big one machine can get, and big machines are expensive. Horizontal scaling is possible with techniques like replication and sharding, but it is difficult because JOINs and transactions across many servers become complicated.

NoSQL: Most NoSQL databases were built for horizontal scaling from the start. Data is automatically split and spread over many cheap servers, and new servers can be added when needed. This is why they are popular with web-scale applications.

4.2 Consistency

Consistency means that every user sees the same, correct data at the same time. This matters a lot in some situations. If I transfer money to a friend, the amount must leave my account and reach theirs together, never just one of the two.

Relational: Relational databases give strong consistency through ACID transactions. Either the whole transaction succeeds or nothing changes. Constraints like primary keys, foreign keys and NOT NULL also protect the data from wrong entries.

NoSQL: Many NoSQL databases choose eventual consistency. The copies of data on different servers may be slightly different for a short time, but they become the same after a while. This

is explained by the CAP theorem, which says a distributed system can fully guarantee only two out of three things: Consistency, Availability and Partition tolerance. NoSQL systems often pick availability and partition tolerance. Note that this is not true for all of them, because some NoSQL databases (like MongoDB) now also support multi-document transactions, and some let the user choose how strict the consistency should be.

4.3 Performance

Performance depends on the type of work the database is doing, so neither type is faster in every case.

Relational: They perform very well for complex queries that join many tables, filter data and calculate totals or reports. Indexes and query optimizers help a lot. However, performance can drop when there are too many JOINs on very large tables or extremely heavy write traffic on a single server.

NoSQL: They give very fast reads and writes for simple operations such as getting a record by its key, because the related data is often stored together in one place and no JOINs are needed. A key-value store like Redis keeps data in memory and can respond in a fraction of a millisecond. The trade-off is that complex queries and reports across different collections are harder and sometimes slower.

4.4 Use Cases

Relational databases are suitable for: banking and financial systems, accounting, inventory management, hospital and student records, ticket and order management, and any system with well-structured data and strict accuracy needs.

NoSQL databases are suitable for: social media feeds, real-time analytics, caching, session storage, product catalogues with different attributes, IoT sensor data, chat messages, recommendation engines and fraud detection (graph).

4.5 Summary Table

Point	Relational Database	NoSQL Database
Data model	Tables with rows and columns	Document, key-value, column-family or graph
Schema	Fixed, defined in advance	Flexible, can change easily
Scalability	Mainly vertical; horizontal is harder	Designed for horizontal scaling
Consistency	Strong (ACID)	Often eventual (BASE), but configurable in many systems
Query language	SQL (standard)	Different for each database
Relationships / JOINs	Excellent support	Limited or handled in the application
Performance	Best for complex queries and transactions	Best for simple, high-volume reads and writes

Point	Relational Database	NoSQL Database
Typical use cases	Banking, ERP, inventory, bookings	Social media, caching, big data, IoT, catalogues
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra, Neo4j

5. Why Organizations Choose Different Databases

There is no single "best" database. Organizations choose based on what their application really needs. These are the main factors:

- **Type and structure of data:** If the data is neat and every record has the same fields (like customer and payment details), relational fits well. If the data is varied, changes often or is connected like a network, NoSQL may fit better.
- **Need for accuracy:** Where a wrong or half-finished transaction can cause real loss, such as money transfers, ACID transactions are a must.
- **Size of data and traffic:** When data and users grow to a very large scale, spreading the load over many servers becomes important, and NoSQL is often chosen.
- **Speed required:** Applications that need responses in milliseconds, like real-time recommendations or live leaderboards, often use in-memory or key-value databases.
- **Complexity of queries:** If the business needs many reports and analysis across several related tables, SQL is very strong.
- **Speed of development:** A flexible schema lets a team change the data structure quickly while the product is still evolving, for example in a startup.
- **Cost, skills and rules:** Teams usually choose what their developers already know, and some industries have rules about data accuracy and auditing that favour relational systems.

Because of all these factors, many big organizations now use more than one type of database together. This approach is called polyglot persistence, which means using the right database for each job.

6. Real-World Examples

Organization / Sector	Database technology	Why it was chosen
Banks and financial institutions	Relational (Oracle, PostgreSQL, SQL Server)	Money transfers need strict ACID transactions so that balances are always correct and every transaction can be audited.
Amazon (shopping cart)	Key-value (DynamoDB, based on the Dynamo design)	The shopping cart must always be available even if some servers fail, so Amazon preferred availability and eventual consistency. Amazon described this in its Dynamo research paper.

Organization / Sector	Database technology	Why it was chosen
Netflix	Column-family (Apache Cassandra)	Netflix has to handle huge volumes of data spread across regions and wants the service to stay up even when servers fail, which suits Cassandra.
Facebook / Meta	Relational (MySQL) and NoSQL (Cassandra was first created at Facebook)	It shows that one company can use both. Relational for core structured data, and NoSQL for large-scale needs such as inbox search.
E-commerce product catalogues	Document (MongoDB)	Different products have different attributes (a phone has RAM, a shirt has size), so flexible documents are easier than fixed columns.
Websites needing fast response	Key-value in memory (Redis)	Used as a cache and for sessions, so that frequently needed data is returned in milliseconds without hitting the main database.
Fraud detection and social networks	Graph (Neo4j)	The relationships between people, accounts or devices are the main thing, and graph databases follow these connections quickly.

Note: These examples are based on well-known public information about how these companies and technologies are used. Companies change their technology over time, so the exact setup may be different today.

6.1 Example from my own project: Cab Booking System

If my Cab Booking System grew into a large service like a real cab company, I think it would be best to use both types of databases:

Data	Better fit	Reason
Customers, drivers, cars	Relational	Structured data with clear relationships and a fixed format.
Bookings and payments	Relational	Payments need ACID transactions. A booking must never be saved without its correct payment record.
Live driver location updates	NoSQL (key-value such as Redis)	Thousands of location updates come every second and need very fast writes and reads.
Trip history and logs	NoSQL (document or column-family)	Very large volume of data that grows constantly and does not need complex JOINs.
Ride recommendations or fraud checks	NoSQL (graph)	Depends on finding connections between users, drivers and trips.

So the core of my cab system (the five tables) would stay relational, while the fast-moving and very large data would go to NoSQL. This is an example of polyglot persistence in a small, familiar setting.

7. Practical Demonstration with SQL

To make this comparison more practical, I implemented the relational part using SQL on my Cab Booking System from the earlier assignment. The code below uses MySQL-style syntax. I also ran it on a test database to confirm that it works and that the outputs shown are correct. After that I show how the same kind of data would look in NoSQL databases.

7.1 Creating the tables (DDL)

Each table gets a primary key, and the foreign keys connect the tables as in my relational schema. The UNIQUE constraint on Booking_ID in the Payment table makes sure that one booking can have only one payment, which is the 1:1 relationship.

```
CREATE TABLE Customer (
    Customer_ID   VARCHAR(10) PRIMARY KEY,
    Customer_Name VARCHAR(50) NOT NULL,
    Phone         VARCHAR(15),
    Email         VARCHAR(50),
    Address       VARCHAR(100)
);

CREATE TABLE Driver (
    Driver_ID    VARCHAR(10) PRIMARY KEY,
    Driver_Name  VARCHAR(50) NOT NULL,
    Phone        VARCHAR(15),
    License_No   VARCHAR(20) UNIQUE,
    Availability VARCHAR(15)
);

CREATE TABLE Car (
    Car_ID      VARCHAR(10) PRIMARY KEY,
    Car_Number  VARCHAR(15) UNIQUE,
    Car_Model   VARCHAR(30),
    Car_Type    VARCHAR(20),
    Fare_Per_KM DECIMAL(6,2),
    Driver_ID   VARCHAR(10),
    FOREIGN KEY (Driver_ID) REFERENCES Driver(Driver_ID)
);

CREATE TABLE Booking (
    Booking_ID      VARCHAR(10) PRIMARY KEY,
    Customer_ID     VARCHAR(10) NOT NULL,
    Car_ID          VARCHAR(10) NOT NULL,
    Booking_Date    DATE,
    Pickup_Location VARCHAR(50),
    Drop_Location   VARCHAR(50),
    Distance_KM     INT,
    Booking_Status  VARCHAR(15),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
    FOREIGN KEY (Car_ID) REFERENCES Car(Car_ID)
);

CREATE TABLE Payment (
    Payment_ID     VARCHAR(10) PRIMARY KEY,
    Booking_ID     VARCHAR(10) UNIQUE NOT NULL,
    Amount         DECIMAL(8,2),
    Payment_Date   DATE,
    Payment_Method VARCHAR(10),
    Payment_Status VARCHAR(10),
```

```
    FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID)
);
```

7.2 Inserting the sample data (DML)

```
INSERT INTO Customer VALUES
('CU01','Rahul','9876543210','rahul@mail.com','Bangalore'),
('CU02','Arjun','9876543211','arjun@mail.com','Chennai'),
('CU03','Kiran','9876543212','kiran@mail.com','Krishnagiri');

INSERT INTO Driver VALUES
('D01','Ravi','9123456780','DL1001','Available'),
('D02','Kumar','9123456781','DL1002','Busy'),
('D03','Suresh','9123456782','DL1003','Available');

INSERT INTO Car VALUES
('C01','KA01AB1234','Swift','Hatchback',12,'D01'),
('C02','KA02CD5678','City','Sedan',15,'D02'),
('C03','KA03EF9012','Creta','SUV',20,'D03');

INSERT INTO Booking VALUES
('B01','CU01','C01','2026-09-07','Airport','MG Road',25,'Completed'),
('B02','CU02','C02','2026-09-07','Mall','Railway Station',12,'Completed'),
('B03','CU03','C03','2026-09-08','College','Airport',30,'Confirmed');

INSERT INTO Payment VALUES
('P01','B01',300,'2026-09-07','UPI','Paid'),
('P02','B02',180,'2026-09-07','Card','Paid'),
('P03','B03',600,'2026-09-08','UPI','Pending');
```

The tables must be created and filled in this order (Customer, Driver, Car, Booking, Payment), because a foreign key can only point to a row that already exists.

7.3 Queries

Query 1: Find the drivers who are available

```
SELECT Driver_ID, Driver_Name, Phone
FROM Driver
WHERE Availability = 'Available';
```

Driver_ID	Driver_Name	Phone
D01	Ravi	9123456780
D03	Suresh	9123456782

Query 2: Show full booking details by joining four tables

```
SELECT b.Booking_ID, c.Customer_Name, car.Car_Model, d.Driver_Name,
       b.Pickup_Location, b.Drop_Location, b.Distance_KM
FROM Booking b
JOIN Customer c ON b.Customer_ID = c.Customer_ID
JOIN Car car    ON b.Car_ID      = car.Car_ID
JOIN Driver d   ON car.Driver_ID = d.Driver_ID;
```

Booking_ID	Customer	Car_Model	Driver	Pickup	Drop	Distance_KM
B01	Rahul	Swift	Ravi	Airport	MG Road	25
B02	Arjun	City	Kumar	Mall	Railway Station	12
B03	Kiran	Creta	Suresh	College	Airport	30

Query 3: Calculate the fare (distance × fare per km) and compare it with the amount paid

```
SELECT b.Booking_ID, b.Distance_KM, car.Fare_Per_KM,
       b.Distance_KM * car.Fare_Per_KM AS Calculated_Fare,
       p.Amount
FROM Booking b
JOIN Car car    ON b.Car_ID = car.Car_ID
JOIN Payment p  ON p.Booking_ID = b.Booking_ID;
```

Booking_ID	Distance_KM	Fare_Per_KM	Calculated_Fare	Amount
B01	25	12	300	300
B02	12	15	180	180
B03	30	20	600	600

Query 4: Total payments grouped by status (aggregate function)

```
SELECT Payment_Status,
       COUNT(*)    AS Total_Payments,
       SUM(Amount) AS Total_Amount
FROM Payment
GROUP BY Payment_Status;
```

Payment_Status	Total_Payments	Total_Amount
Paid	2	480
Pending	1	600

Query 5: Update records when the trip is completed and the payment is received

```
UPDATE Payment SET Payment_Status = 'Paid'      WHERE Payment_ID = 'P03';
UPDATE Booking SET Booking_Status = 'Completed' WHERE Booking_ID = 'B03';
```

7.4 Showing ACID: a transaction

When a new booking is made, the booking row and its payment row must be saved together. If the second step fails, the first one should be undone. This is exactly what a relational transaction does, and it is the strong consistency that I described in Section 4.2.

```
START TRANSACTION;

INSERT INTO Booking VALUES
('B04','CU01','C03','2026-09-09','Mall','College',18,'Confirmed');
INSERT INTO Payment VALUES
('P04','B04',360,'2026-09-09','Cash','Pending');
```

```
COMMIT;      -- both rows are saved permanently
-- If anything goes wrong, use ROLLBACK; instead of COMMIT;
-- and neither the booking nor the payment will be saved.
```

7.5 Showing integrity: a foreign key violation

Now I try to insert a booking for a customer that does not exist. The database rejects it, so wrong data cannot enter the table. A NoSQL database with a flexible schema would normally not stop this kind of mistake by itself, and the application would have to check it.

```
INSERT INTO Booking VALUES ('B05', 'CU99', 'C01', '2026-09-10', 'A', 'B', 5,
'Confirmed');
-- Result: ERROR - foreign key constraint fails (Customer CU99 does not exist)
```

7.6 The same booking data in NoSQL

For comparison, here is how the same kind of data could be stored in NoSQL databases. These examples are written in the standard MongoDB shell and Redis command styles. I did not run these two, so they are for understanding the idea only.

MongoDB (document database): One booking is stored as a single document that already contains the customer, car and payment details. So there are no JOINs, but the customer and car data is repeated inside each booking.

```
db.bookings.insertOne({
  _id: "B01",
  booking_date: "2026-09-07",
  customer: { id: "CU01", name: "Rahul", city: "Bangalore" },
  car: { id: "C01", model: "Swift", fare_per_km: 12 },
  pickup: "Airport",
  drop: "MG Road",
  distance_km: 25,
  status: "Completed",
  payment: { id: "P01", amount: 300, method: "UPI", status: "Paid" }
});

// Find all bookings of one customer
db.bookings.find({ "customer.id": "CU01" });
```

Redis (key-value database): Good for very fast, frequently changing data like the current status and location of a driver.

```
HSET driver:D01 status "Available" lat 12.9716 lng 77.5946
HGET driver:D01 status
HGETALL driver:D01
```

7.7 SQL vs NoSQL: what I observed

Point	SQL (relational)	NoSQL (MongoDB / Redis)
Getting booking details	Needs JOINs across four tables	One read gets everything, because data is stored together

Point	SQL (relational)	NoSQL (MongoDB / Redis)
Data duplication	Very little. Each fact is stored once.	More duplication, because customer and car data is copied into each booking
Wrong data	Blocked by constraints and foreign keys	Must be checked by the application
Saving booking and payment together	Easy with a transaction (COMMIT / ROLLBACK)	Possible in some systems, but not the main strength
Reports and totals	Very easy with GROUP BY and SUM	Possible but usually more work

8. Conclusion

From this study I learned that relational and NoSQL databases are not enemies, and writing the SQL for my own cab system showed me how constraints and transactions protect the data. They are tools built for different jobs. Relational databases are best when data is structured, relationships are important and accuracy is critical, because of strong consistency and the power of SQL. NoSQL databases are best when the data is huge, varied or changing quickly, and when the system must scale across many servers and stay fast and available.

Organizations choose their databases by looking at their data type, accuracy needs, traffic, speed requirements, query complexity, team skills and cost. Real companies often combine both types instead of choosing only one. So the right question is not "Which database is better?" but "Which database is right for this particular requirement?"